MODUL 7 · DATA FETCHING & CACHING

Server Components vs Client Components



Kapan pakai yang mana, dengan cara yang gampang diingat

Dua Jenis Komponen di Next.js

Keduanya komponen React biasa. Bedanya cuma satu: dijalankan di mana.

Server Component

Bawaan (default)

- Semua page.tsx dan layout.tsx di App Router otomatis jadi Server Component
- Tidak perlu menulis apa-apa

Client Component

Ditandai 'use client'

- Taruh 'use client' di baris paling atas file
- Kita yang memilih hanya kalau memang perlu

Perbedaan nya adalah : cara kerja dan apa yang boleh dipakai.

Analogi: Pesan Makanan

Supaya gampang kebayang, anggap server itu dapur dan browser itu meja kamu.

Server Component

Makanan sudah jadi

- Semua dimasak di dapur (server)
- Kamu terima piringnya yang sudah matang
- Bahan dan resep rahasia tetap di dapur

Client Component

Bahan dan resep dikirim

- Dimasak di meja kamu (perangkat pengguna)
- Kode JavaScript-nya dikirim ke browser
- Bisa diubah-ubah sesuai pilihanmu (interaktif)

Cara Kerja Server Component

UI dibuat di server

Server menjalankan komponennya, lalu mengirim hasil jadinya ke browser, bukan di-generate di client.

Kodenya tidak ikut dikirim

Browser menerima hasil, bukan kodenya, jadi JavaScript yang diunduh lebih sedikit.

Akses database dan secret langsung

Karena berjalan di server, API key dan query database tidak terlihat pengguna.

Tidak bisa menggunakan Hook

Server tidak tahu pengguna mengklik apa. Interaksi hanya terjadi di browser.

Cara Kerja Client Component

Kodenya dikirim ke browser dan dijalankan di sana

Makanya bisa bereaksi saat pengguna klik, ketik, atau scroll.

Load pertama: tetap ada HTML awal

Next.js membuat HTML awalnya dulu supaya cepat tampil, lalu browser menghidupkannya (***hydration***).

Cocok untuk state, event, effect, dan browser API

useState, onClick, useEffect, localStorage, window.

Contoh Server Component

Tidak ada 'use client', dan komponennya boleh async: data diambil langsung.

app/page.tsx

```
// Server Component: bawaan, tanpa 'use client'
export default async function Page() {
  const res = await fetch("https://api.contoh.com/post/1")
  const post = await res.json()

  return <h1>{post.title}</h1>
}
```

Contoh Client Component

Ada useState dan onClick, jadi harus Client Component.

app/like-button.tsx

```
'use client'
import { useState } from 'react'

export default function LikeButton() {
  const [likes, setLikes] = useState(0)
  return (
    <button onClick={() => setLikes(likes + 1)}>
      {likes} suka
    </button>
  )
}
```

Kenapa Server Component Lebih Nyaman untuk SEO?

Search Engine akan read HTML, tapi... apakah konten kita sudah render semua atau belum?

Server: all in one

Satu paket lengkap

- Seluruh halaman dirakit di server
- Dikirim sebagai satu paket HTML yang sudah lengkap

Client: satu per satu

Bertahap di browser

- Tiap komponen render dan fetch datanya sendiri
- Konten muncul bertahap setelah JavaScript jalan

Catatan: Client Component tetap dibuatkan HTML awal

Tapi data yang diambil di browser belum ada di HTML itu. Yang ada baru **kerangka.**

Ilustrasi: HTML yang Diterima Crawler

Halaman yang sama, judul yang sama. Hanya beda komponennya. Ilustrasi disederhanakan dari hasil View Source.

Server Component

```
<body>
  <h1>
    Belajar Next.js
  </h1>
</body>
```

Client Component (fetch di useEffect)

```
<body>
  <h1>
    Memuat...
  </h1>
</body>
```

Perbandingan Singkat

Aspek	Server Component	Client Component
Dijalankan di	Server	Browser (HTML awal tetap dibuat di server)
useState, onClick, useEffect	Tidak bisa	Bisa
Akses database dan secret	Bisa langsung	Tidak boleh (bisa bocor ke browser)
JavaScript ke browser	Kode komponen tidak dikirim	Kode dikirim dan dijalankan
Mengambil data	async/await langsung	useEffect atau library (video berikutnya)

Kapan Pakai yang Mana?

Daftar resmi dari dokumentasi Next.js.

Client Component

Ada interaksi atau butuh browser

- State dan event handler (useState, onClick, onChange)
- Lifecycle (useEffect)
- Browser API (localStorage, window)
- Custom hooks

Server Component

Menampilkan dan mengambil data

- Mengambil data dari database atau API
- Memakai API key dan secret dengan aman
- Mengurangi JavaScript ke browser
- Tampil lebih cepat dan bisa di-streaming

Cara Gampang Mengingatnya

Bisa diklik, diketik, atau berubah-ubah?

Ada useState, onClick, useEffect, atau butuh window? Berarti Client Component.

Cuma menampilkan data?

Berarti Server Component. Ini default-nya, tidak perlu ditulis apa-apa.

Mulai dari Server

Tambahkan 'use client' hanya saat benar-benar butuh, dan di file sekecil mungkin.

Jebakan yang Sering Terjadi

'use client' di paling atas page

Semua yang diimpor ikut jadi client dan manfaat Server Component hilang. Pisahkan bagian interaktif ke file kecil sendiri.

Props ke Client Component harus bisa *serialized*

Teks, angka, objek, dan array boleh. Fungsi biasa tidak bisa dikirim dari Server ke Client.

Context tidak jalan di Server Component

Bungkus provider-nya dalam Client Component yang menerima children.